

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Чувашский государственный университет им. И.Н. Ульянова»
Кафедра вычислительной техники

Лабораторная работа № 5
По дисциплине «Операционные системы»

Выполнил: студент группы ИВТ-42-23
Иванов И.Л.

Проверил: кандидат технических наук
Ковалев С.В.

Чебоксары, 2026

Цель: Реализовать метод обработки прерываний и регулировщик применительно к ПО «Модель ОС».

Исходники: https://github.com/kukirmash/os_model/

Задание

1. Доделать подпрограмму «Выполнение операции ввода (вывода)».
 2. Реализовать индивидуальный алгоритм планировщика в виде процедуры.
 3. Реализовать подпрограмму работы процессоров ввода (вывода).
 4. Реализовать моделирование сигнала прерывания по окончании операции ввода (вывода).
 5. Реализовать проверку состояния системы.
 6. Обновить подпрограмму «Выполнение одного такта моделирования».
 7. Обновить подпрограмму индикации.
- Получить результаты:
- формулы выбора следующего процесса для обслуживания на ЦПр;
 - индикацию параметров в соответствии с индивидуальным алгоритмом работы планировщика.

БСА

Алгоритмы, реализованные в данной работе, представлены на рисунках 1–4.

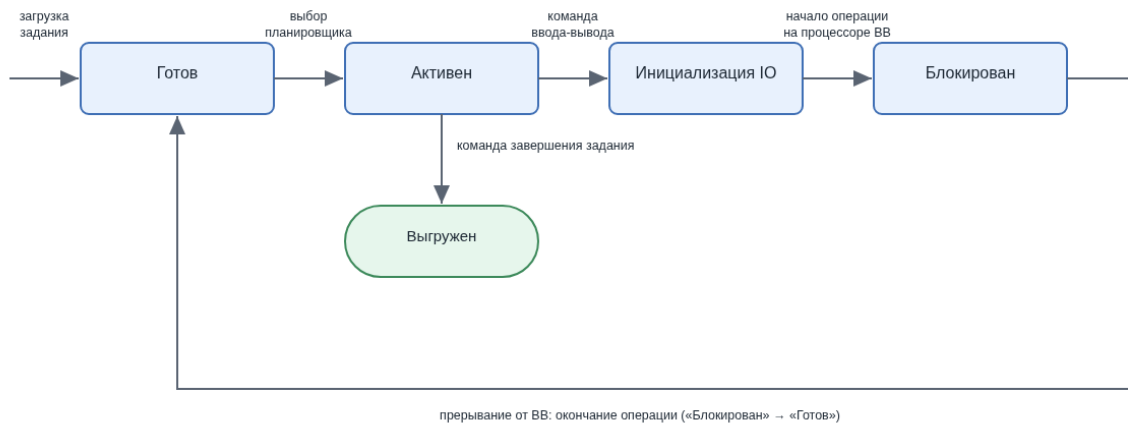


Рисунок 1 — Граф состояний процесса и события регулятора

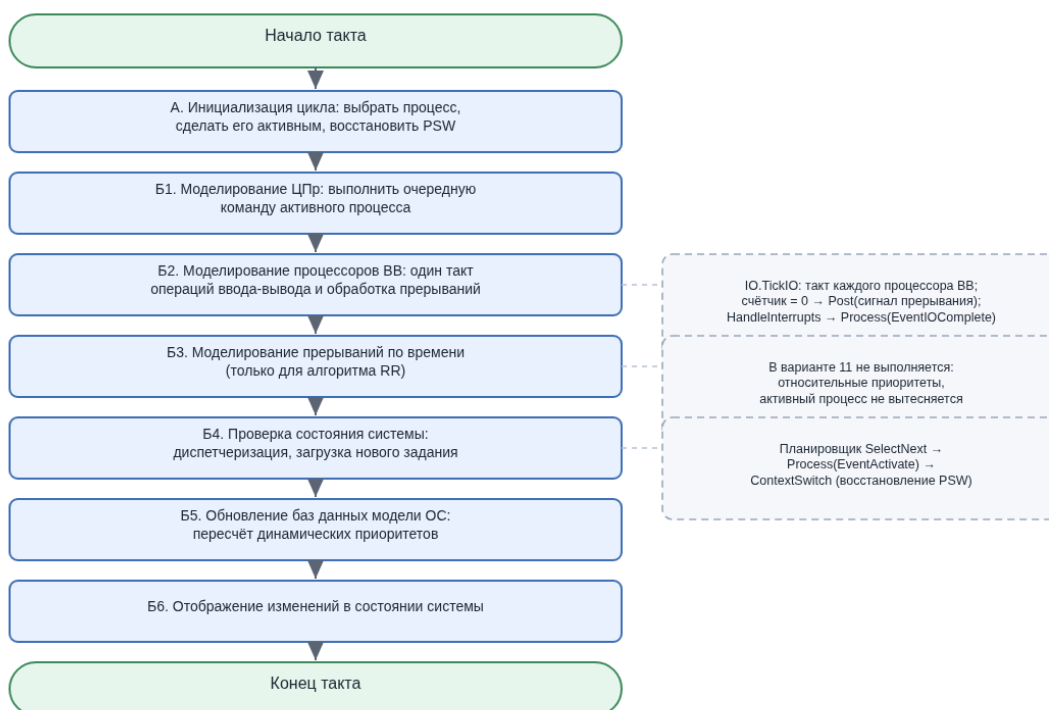


Рисунок 2 — Блок-схема одного такта моделирования (А, B1–B6)

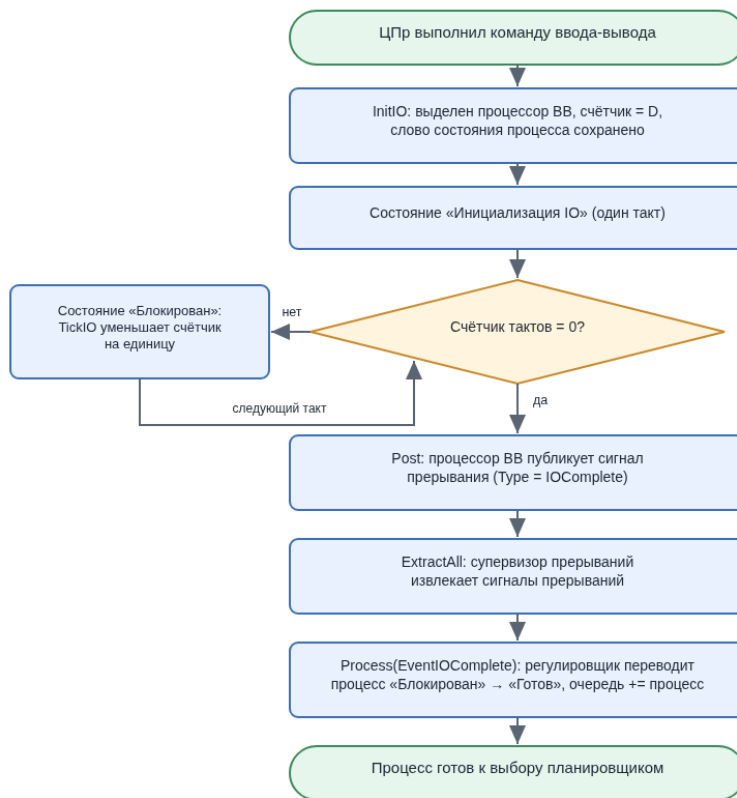


Рисунок 3 — Блок-схема обработки операции ввода-вывода и сигнала прерывания

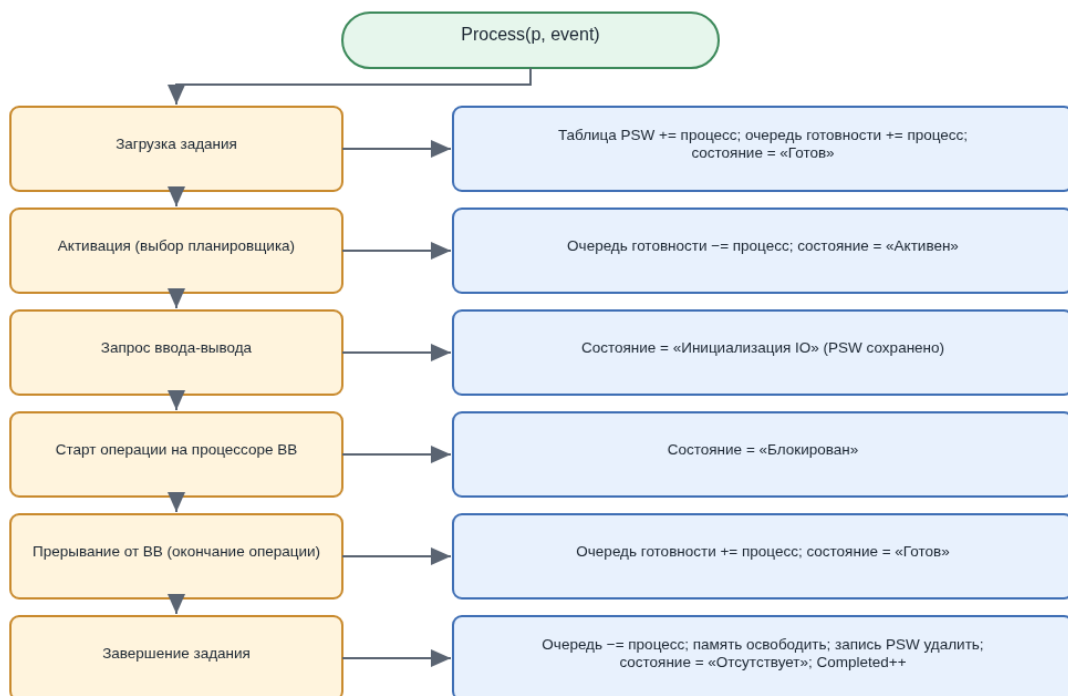


Рисунок 4 — Подпрограмма регулятора Process(p, event)

Анализ программного кода

1. Состояния процесса (internal/process/psw.go).

Реализован полный список состояний, требуемый лабораторной работой: «Готов», «Активен», «Инициализация IO», «Блокирован». Служебное состояние «Отсутствует» (StateNone) применяется при выгрузке процесса, когда запись удаляется из таблицы. Слово состояния процесса (PSW) содержит только собственные данные процесса: идентификатор, ссылку на задание, счётчик команд, состояние и динамический приоритет. Все изменения поля State выполняет исключительно регулятор (подпрограмма Process), остальные модули лишь читают это поле.

```
// State – состояние процесса в модели.
//
// Список возможных состояний по лабораторной работе 5:
// «Готов», «Активен», «Инициализация IO», «Блокирован».
// StateNone используется при выгрузке процесса (запись в таблице удаляется).
// PSW (Process Status Word, слово состояния процесса) описывает
// динамический процесс, порождённый заданием.
//
// Все изменения состояния процесса выполняет только регулятор
// (подпрограмма Process), остальные модули лишь читают эти поля.
// Ход операции ввода-вывода (номер процессора ВВ и счётчик тактов)
// хранится в состоянии процессора ВВ – см. пакет io.
type PSW struct {
    ID          int    // идентификатор процесса
    Task        *Task  // ссылка на задание, породившее процесс
    PC          int    // счётчик команд
    State       State  // текущее состояние
    DynamicPriority int  // текущий (динамический) приоритет
}
```

2. Регулятор (internal/regulator/regulator.go).

Регулятор — часть ОС, отвечающая за состояния процессов. По теории только регулятор имеет право изменять состояния процессов (в противном случае возможно возникновение хаоса или «потеря» процессов). Регулятор реализован в виде подпрограммы Process(p, event), которая принимает процесс и событие модели: загрузка задания, активация планировщиком, запрос ввода-вывода, старт операции на процессоре ВВ, окончание ввода-вывода (прерывание) и завершение задания. Подпрограмма выполняет переход процесса в новое состояние и связанные с этим изменения общих данных модели: таблицы слов состояний процессов, очереди готовности, памяти и счётчика выполненных заданий. Недопустимые для текущего состояния события отклоняются.

Дополнительно реализована подпрограмма Terminate, вызываемая центральным процессором при выполнении команды завершения задания.

```
    EventLoad          Event = iota // загрузка задания: создан процесс в состоянии
«Готов»
    EventActivate      // выбор планировщика: «Готов» → «Активен»
    EventIORequest     // инициализация ВВ: «Активен» → «Инициализация
IO»
    EventIOStart       // процессор ВВ начал работу: «Инициализация IO» →
«Блокирован»
    EventIOComplete    // прерывание от ВВ: «Блокирован» → «Готов»
    EventTerminate     // завершение задания: выгрузка процесса
)
// Process – основная подпрограмма регулятора: обрабатывает событие event
// для процесса p, изменяя его состояние и общие данные модели.
// Возвращает false, если событие недопустимо для текущего состояния процесса.
func (r *Regulator) Process(p *process.PSW, event Event) bool {
    switch event {
    case EventLoad:
        // Загрузка задания: нужна свободная запись в таблице PSW.
        if !r.FreeSlot() {
            return false
        }
        p.State = process.StateReady
        r.Table = append(r.Table, p)
        r.Planner.Add(p)

    case EventActivate:
        // Планировщик выбрал процесс для выполнения на процессоре.
        if p.State != process.StateReady {
            return false
        }
        p.State = process.StateActive
        r.Planner.Remove(p)

    case EventIORequest:
        // Задание инициализирует операцию ввода-вывода: слово состояния
        // процесса сохраняется, процесс переводится в «Инициализация IO».
        if p.State != process.StateActive {
            return false
        }
        p.State = process.StateInitIO

    case EventIOStart:
        // Процессор ВВ приступил к выполнению операции.
        if p.State != process.StateInitIO {
            return false
        }
        p.State = process.StateBlockedIO

    case EventIOComplete:
        // Сигнал прерывания от процессора ВВ: процесс снова готов
        // к выполнению и помещается в очередь готовности.
        if p.State != process.StateBlockedIO {
            return false
        }
        p.State = process.StateReady
        r.Planner.Add(p)

    case EventTerminate:
        // Завершение задания: выгрузка процесса.
        if p.State != process.StateActive {
            return false
        }
        r.Planner.Remove(p)
        r.Memory.Free(*p.Task)
```

```

        r.removeFromTable(p)
        p.State = process.StateNone
        r.Completed++

    default:
        return false
    }

    return true
}

```

3. Планировщик (`internal/regulator/planner.go`).

Планировщик — часть регуляровщика, поэтому его модуль перенесён в пакет `regulator`. Планировщик решает только одну задачу: выбор следующего процесса из списка готовности, и не изменяет состояния процессов. Индивидуальный алгоритм варианта 11 — относительные динамические приоритеты. Приоритет учитывается только в момент выбора, активный процесс не вытесняется новым поступлением: выбор выполняется лишь при освобождении центрального процессора. Динамический приоритет растёт со временем ожидания. Процедура выбора `SelectNext` возвращает процесс с максимальным динамическим приоритетом (при равенстве — стоящий раньше в очереди) и не изымает его из очереди: добавление и исключение процессов выполняет регуляровщик методами `Add` и `Remove`. Пересчёт приоритетов выполняется каждый такт методом `UpdatePriorities`: прирост приоритета пропорционален размеру задания.

```

// SelectNext выбирает процесс с наивысшим DynamicPriority, не изменяя
// ни очередь, ни состояния процессов.
func (p *Planner) SelectNext() *process.PSW {
    if len(p.ReadyQueue) == 0 {
        return nil
    }

    // Формула выбора: процесс с максимальным DynamicPriority;
    // при равенстве приоритетов выбирается стоящий раньше в очереди.
    best := 0
    for i, psw := range p.ReadyQueue {
        if psw.DynamicPriority > p.ReadyQueue[best].DynamicPriority {
            best = i
        }
    }
    return p.ReadyQueue[best]
}

// UpdatePriorities пересчитывает приоритеты процессов в очереди готовности.
// Вызывается каждый такт (шаг Б5 алгоритма такта).
//
// Вариант 11: прирост DynamicPriority тем больше, чем больше размер задания
// (PSW.Task.Size) — приоритет больших заданий увеличивается сильнее.
func (p *Planner) UpdatePriorities() {
    for _, psw := range p.ReadyQueue {

```

```

        // Прирост за такт ожидания пропорционален размеру задания.
        psw.DynamicPriority += psw.Task.Size
    }
}

```

4. Подсистема ввода-вывода (internal/io/processors.go).

По допущению лабораторных работ количество процессоров ввода-вывода достаточно для работы без очередей, поэтому каждая операция выполняется на отдельном (условном) процессоре ВВ и описывается структурой Operation: номер процессора ВВ, обслуживаемый процесс, оставшиеся и полные такты операции. Подпрограмма инициализации InitIO вызывается центральным процессором при выполнении команды ВВ: выделяется процессор ВВ, слово состояния процесса сохраняется, и регулятор переводит процесс в состояние «Инициализация IO». Подпрограмма работы процессоров ВВ TickIO выполняется каждый такт: такт, в котором операция создана, целиком уходит на инициализацию; на первом такте обслуживания процесс переводится в «Блокирован», после чего счётчик уменьшается. По достижении нуля процессор ВВ публикует сигнал прерывания. Состояния процессов здесь не изменяются — только через регулятора.

```

// Operation – операция ввода-вывода, выполняемая на процессоре ВВ.
type Operation struct {
    ProcessorID int // номер процессора ВВ
    Process     *process.PSW // обслуживаемый процесс
    TicksLeft   int // осталось тактов выполнения
    TotalTicks  int // полная длительность операции

    initialized bool // такт инициализации пройден
    started      bool // процесс переведён в состояние «Блокирован»
}
// InitIO – подпрограмма инициализации операции ввода-вывода: выделяет
// процессор ВВ, сохраняет слово состояния процесса (через регулятора)
// и переводит его в состояние «Инициализация IO».
func (m *Processors) InitIO(p *process.PSW, duration int) {
    m.nextProcessor++

    // Смена состояния – только через регулятора.
    m.Regulator.Process(p, regulator.EventIORequest)

    m.Operations = append(m.Operations, &Operation{
        ProcessorID: m.nextProcessor,
        Process:     p,
        TicksLeft:   duration,
        TotalTicks:  duration,
    })
}
// TickIO – подпрограмма работы процессоров ввода-вывода: выполняет один такт

```



```

// каждой операции, уменьшает счётчики тактов и по окончании операции
// публикует сигнал прерывания. Состояния процессов не изменяются.
func (m *Processors) TickIO(now int) {
    busy := m.Operations[:0]
    for _, op := range m.Operations {
        // Такт, в котором операция инициализирована, целиком уходит
        // на инициализацию: процессор ВВ принимает операцию.
        if !op.initialized {
            op.initialized = true
            busy = append(busy, op)
            continue
        }

        // Первый такт обслуживания: «Инициализация IO» → «Блокирован».
        if !op.started {
            op.started = true
            m.Regulator.Process(op.Process, regulator.EventIOStart)
        }

        op.TicksLeft--
        if op.TicksLeft > 0 {
            busy = append(busy, op)
            continue
        }

        // Счётчик тактов равен нулю — процессор ВВ публикует прерывание.
        m.Interrupts.Post(interrupt.Signal{
            Type:    interrupt.IOComplete,
            Time:    now,
            Process:  op.Process,
            SourceID: op.ProcessorID,
        })
    }
    m.Operations = busy
}

```

5. Супервизор прерываний (internal/interrupt/interrupt.go).

Сигнал прерывания описывается структурой `Signal`: тип прерывания, такт модели, процесс и номер источника (процессора ВВ). Определены типы `IOComplete` — прерывание по окончании операции ввода-вывода — и `Timer` — прерывание от таймера, которое в варианте 11 не используется, так как вытеснение по кванту времени не применяется (алгоритм не RR); тип оставлен как задел. Источник публикует сигнал методом `Post`: сигнал попадает в очередь ожидающих обработки и в журнал последних сигналов для индикации. Супервизор прерываний ядро модели вызывает каждый такт: метод `ExtractAll` извлекает все накопившиеся сигналы, после чего обработчик вызывает регулировщика `Process(EventIOComplete)`, который переводит процесс из состояния «Блокирован» в «Готов» и ставит его в очередь готовности.

```

IOComplete Type = iota // окончание операции ввода-вывода

```

```

    Timer                // прерывание от таймера (в варианте 11 не используется)
// Signal – смоделированный сигнал прерывания.
type Signal struct {
    Type    Type    // тип прерывания
    Time    int     // такт модели, в котором возникло прерывание
    Process *process.PSW // процесс, которого касается прерывание
    SourceID int     // номер источника (процессора ввода-вывода)
}
// Post – источник публикует сигнал прерывания.
func (s *Supervisor) Post(signal Signal) {
    s.Pending = append(s.Pending, signal)
    s.Log = append(s.Log, signal)
    if len(s.Log) > logLimit {
        s.Log = s.Log[len(s.Log)-logLimit:]
    }
}
// ExtractAll извлекает все накопившиеся сигналы для обработки.
func (s *Supervisor) ExtractAll() []Signal {
    pending := s.Pending
    s.Pending = nil
    s.Handled += len(pending)
    return pending
}

```

6. Проверка состояния системы и такт моделирования (internal/os/system.go).

Подпрограмма выполнения одного такта построена строго по алгоритму лабораторной работы. А — инициализация цикла: если центральный процессор свободен, планировщик выбирает процесс, регулировщик переводит его в «Активен», процессор восстанавливает слово состояния. Б1 — моделирование ЦПр: выполняется очередная команда активного процесса. Б2 — моделирование процессоров ВВ: один такт каждой операции и обработка прерываний от процессоров ВВ. Б3 — моделирование прерываний по времени — в варианте 11 отсутствует, так как используются относительные приоритеты без вытеснения. Б4 — проверка достигнутого состояния системы. Б5 — обновление баз данных модели (пересчёт динамических приоритетов). Б6 — отображение изменений в состоянии системы.

Проверка состояния системы (шаг Б4) реализована отдельной подпрограммой CheckSystemState. Если резидентный процесс перестал быть активным (заблокирован или завершён), процессор освобождается. Если процессор находится в состоянии «Ожидание», а список готовности не пуст, вызывается планировщик. Если готовых процессов нет, процессор переходит

в «Ожидание», а ОС порождает системный процесс загрузки нового задания при наличии ресурсов: свободной записи в таблице слов состояний процессов и достаточной памяти (GenerateAndLoadTask). Диспетчеризация dispatchProcess связывает планировщик и регулировщик: SelectNext выбирает процесс, регулировщик переводит его в «Активен», ContextSwitch процессора восстанавливает слово состояния.

```
// DoOneTick – выполнение одного такта моделирования в соответствии
// с алгоритмом одного такта модели ОС:
//
// A – инициализация цикла: выбрать процесс, сделать его активным
// и восстановить слово состояния процесса;
// B1 – моделирование ЦПр: выполнить очередную команду активного процесса;
// B2 – моделирование процессоров ВВ: один такт операций ввода-вывода
// и обработка прерываний от процессоров ВВ;
// B3 – моделирование прерываний по времени (только для алгоритма RR;
// вариант 11 использует относительные приоритеты без вытеснения,
// поэтому шаг отсутствует);
// B4 – проверка достигнутого состояния системы и вызов планировщика;
// B5 – обновление баз данных модели ОС;
// B6 – отображение изменений в состоянии системы.
func (sys *System) DoOneTick() {
    sys.Time++

    // A. Инициализация цикла.
    if sys.CPU.State == cpu.StateIdle {
        sys.dispatchProcess()
    }

    // B1. Моделирование центрального процессора.
    sys.CPU.Tick()

    // B2. Моделирование процессоров ввода-вывода и обработка прерываний.
    sys.IO.TickIO(sys.Time)
    sys.HandleInterrupts()

    // B3. Прерывания по времени не моделируются (см. комментарий выше).

    // B4. Проверка достигнутого состояния системы.
    sys.CheckSystemState()

    // B5. Обновление баз данных модели ОС.
    sys.Regulator.Planner.UpdatePriorities()

    // B6. Отображение изменений в состоянии системы.
    sys.notify()
}
// CheckSystemState – проверка достигнутого состояния системы (шаг B4).
//
// Для всех процессоров: если резидентный процесс перестал быть активным
// (заблокирован или завершён), процессор освобождается; если процессор
// находится в состоянии «Ожидание», а список готовности не пуст, вызывается
// планировщик для выбора следующего процесса. Если готовых процессов нет,
// процессор переходит в состояние «Ожидание», а ОС порождает системный
// процесс загрузки нового задания, если есть ресурсы.
func (sys *System) CheckSystemState() {
    // Резидентный процесс мог перестать быть активным: его состояние
    // изменил регулировщик по событию ВВ или завершения.
    if p := sys.CPU.ActiveProcess; p != nil && p.State != process.StateActive {
        sys.CPU.Release()
    }
}
```

```

}

if sys.CPU.State != cpu.StateIdle {
    return
}

// Список готовности не пуст – планировщик выбирает следующий процесс.
if len(sys.Regulator.Planner.ReadyQueue) > 0 {
    sys.dispatchProcess()
    return
}

// Готовых процессов нет: ЦПр «Ожидает», ОС может загрузить одно новое
// задание (место в таблице PSW и достаточная память проверяются
// в GenerateAndLoadTask).
if sys.GenerateAndLoadTask() {
    sys.dispatchProcess()
}
}

// HandleInterrupts – супервизор прерываний: обрабатывает накопленные сигналы.
// Обработка сводится к вызову регулировщика, который изменяет состояния
// процессов (прерывание от процессора ВВ переводит процесс в «Готов»).
func (sys *System) HandleInterrupts() {
    for _, signal := range sys.Interrupts.ExtractAll() {
        switch signal.Type {
            case interrupt.IOComplete:
                sys.Regulator.Process(signal.Process, regulator.EventIOComplete)
        }
    }
}
}

```

7. Индикация (internal/os/snapshot.go, frontend/dist).

Снимок состояния модели дополнен данными, требуемыми лабораторной работой: очередь готовности с динамическими приоритетами, список занятых процессоров ВВ (номер процессора, процесс, состояние, остаток и полная длительность операции) и журнал обработанных прерываний (такт, тип, источник, процесс). В интерфейсе появились карточки «Процессоры ввода-вывода» и «Прерывания», состояния процессов выводятся цветными метками, включая «Инициализация IO». В подвале интерфейса отображаются формулы алгоритма планировщика. Снимок отправляется в интерфейс каждый такт (шаг Б6).

```

// OperationInfo – данные одной операции ввода-вывода (занятого процессора ВВ).
type OperationInfo struct {
    ProcessorID int `json:"processorId"` // номер процессора ВВ
    ProcessID   int `json:"processId"`   // обслуживаемый процесс
    State       int `json:"state"`       // «Инициализация IO» или «Блокирован»
    TicksLeft   int `json:"ticksLeft"`   // осталось тактов
    TotalTicks  int `json:"totalTicks"`  // полная длительность операции
}

// InterruptInfo – данные сигнала прерывания для индикации.
type InterruptInfo struct {
    Type      int `json:"type"`      // тип прерывания
    Time      int `json:"time"`      // такт модели
    ProcessID int `json:"processId"` // процесс, которого касается прерывание
}

```

```

    SourceID int `json:"sourceId"` // номер источника (процессора ВВ)
}

```

8. Автоматическая проверка модели (internal/os/system_test.go).

В работе добавлен тест, который прогоняет модель на несколько тысяч тактов и проверяет согласованность данных: все процессы находятся в допустимых состояниях, число готовых процессов совпадает с длиной очереди готовности, число процессов на ВВ совпадает с числом операций, занятая память совпадает с суммой размеров резидентных заданий, все сигналы прерываний обработаны, а число загруженных заданий равно сумме выполненных и резидентных. Тест подтверждает корректность регулировщика и супервизора прерываний.

Листинг программы

Файл internal/process/psw.go:

```

package process

// State – состояние процесса в модели.
//
// Список возможных состояний по лабораторной работе 5:
// «Готов», «Активен», «Инициализация IO», «Блокирован».
// StateNone используется при выгрузке процесса (запись в таблице удаляется).
type State int

const (
    StateNone      State = iota // процесс не порождён / выгружен
    StateReady      // «Готов»: находится в очереди планировщика
    StateActive     // «Активен»: выполняется на центральном процессоре
    StateInitIO     // «Инициализация IO»: инициализируется операция
    ввода-вывода
    StateBlockedIO // «Блокирован»: ожидает окончания операции ввода-
    вывода
)

// PSW (Process Status Word, слово состояния процесса) описывает
// динамический процесс, порождённый заданием.
//
// Все изменения состояния процесса выполняет только регулировщик
// (подпрограмма Process), остальные модули лишь читают эти поля.
// Ход операции ввода-вывода (номер процессора ВВ и счётчик тактов)
// хранится в состоянии процессора ВВ – см. пакет io.
type PSW struct {
    ID          int // идентификатор процесса
    Task        *Task // ссылка на задание, породившее процесс
    PC          int // счётчик команд
    State       State // текущее состояние
    DynamicPriority int // текущий (динамический) приоритет
}

```

Файл internal/regulator/regulator.go:

```
// Package regulator реализует регулятор – часть ОС, отвечающую за
// состояния процессов, и включает планировщик (Planner).
//
// Только регулятор имеет право изменять состояния процессов: остальные
// модули модели (центральный процессор, подсистема ввода-вывода) лишь
// обращаются к подпрограмме Process, сообщая о произошедшем событии.
// Планировщик решает одну задачу – выбор следующего процесса из списка
// готовности; состояния он не изменяет.
// Регулятор переводит процесс в новое состояние и выполняет связанные
// с этим изменения общих данных модели: очереди готовности, таблицы слов
// состояний процессов, памяти и счётчика завершённых заданий.
package regulator

import (
    "os_model/internal/memory"
    "os_model/internal/process"
)

// Event – событие модели, по которому регулятор изменяет состояние процесса.
type Event int

const (
    EventLoad          Event = iota // загрузка задания: создан процесс в состоянии
    «Готов»
    EventActivate          // выбор планировщика: «Готов» → «Активен»
    EventIORequest         // инициализация ВВ: «Активен» → «Инициализация
    IO»
    EventIOStart           // процессор ВВ начал работу: «Инициализация IO» →
    «Блокирован»
    EventIOComplete        // прерывание от ВВ: «Блокирован» → «Готов»
    EventTerminate         // завершение задания: выгрузка процесса
)

// Regulator – регулятор модели. Включает планировщик (Planner),
// выбирающий следующий процесс из очереди готовности.
type Regulator struct {
    Planner *Planner // планировщик: очередь готовности и диспетчеризация

    Memory      *memory.Allocator // супервизор памяти: освобождается при
    завершении
    MaxProcesses int      // число записей в таблице слов состояний
    процессов

    Table      []*process.PSW // таблица слов состояний процессов (резидентные
    процессы)
    Completed int           // число выполненных (завершённых) заданий
}

// Process – основная подпрограмма регулятора: обрабатывает событие event
// для процесса p, изменяя его состояние и общие данные модели.
// Возвращает false, если событие недопустимо для текущего состояния процесса.
func (r *Regulator) Process(p *process.PSW, event Event) bool {
    switch event {
    case EventLoad:
        // Загрузка задания: нужна свободная запись в таблице PSW.
        if !r.FreeSlot() {
            return false
        }
        p.State = process.StateReady
        r.Table = append(r.Table, p)
        r.Planner.Add(p)
```

```

case EventActivate:
    // Планировщик выбрал процесс для выполнения на процессоре.
    if p.State != process.StateReady {
        return false
    }
    p.State = process.StateActive
    r.Planner.Remove(p)

case EventIORequest:
    // Задание инициализирует операцию ввода-вывода: слово состояния
    // процесса сохраняется, процесс переводится в «Инициализация IO».
    if p.State != process.StateActive {
        return false
    }
    p.State = process.StateInitIO

case EventIOStart:
    // Процессор ВВ приступил к выполнению операции.
    if p.State != process.StateInitIO {
        return false
    }
    p.State = process.StateBlockedIO

case EventIOComplete:
    // Сигнал прерывания от процессора ВВ: процесс снова готов
    // к выполнению и помещается в очередь готовности.
    if p.State != process.StateBlockedIO {
        return false
    }
    p.State = process.StateReady
    r.Planner.Add(p)

case EventTerminate:
    // Завершение задания: выгрузка процесса.
    if p.State != process.StateActive {
        return false
    }
    r.Planner.Remove(p)
    r.Memory.Free(*p.Task)
    r.removeFromTable(p)
    p.State = process.StateNone
    r.Completed++

default:
    return false
}

return true
}

// Terminate – подпрограмма завершения процесса, используемая процессором
// при выполнении команды завершения задания.
func (r *Regulator) Terminate(p *process.PSW) {
    r.Process(p, EventTerminate)
}

// FreeSlot сообщает, есть ли свободная запись в таблице слов состояний процессов.
func (r *Regulator) FreeSlot() bool {
    return len(r.Table) < r.MaxProcesses
}

// removeFromTable удаляет запись процесса из таблицы слов состояний процессов.
func (r *Regulator) removeFromTable(p *process.PSW) {
    for i, t := range r.Table {

```

```

        if t == p {
            r.Table = append(r.Table[:i], r.Table[i+1:]...)
            return
        }
    }
}

```

Файл internal/regulator/planner.go:

```

package regulator

import "os_model/internal/process"

// Planner – планировщик, часть регулятора (вариант 11).
//
// Решает одну задачу: выбор следующего процесса из списка готовности для
// выполнения на процессоре. Состояния процессов планировщик не изменяет –
// это делает регулятор (подпрограмма Process).
//
// Используются относительные динамические приоритеты: текущий активный
// процесс не вытесняется новым поступлением – выбор следующего процесса
// выполняется только при освобождении процессора.
type Planner struct {
    ReadyQueue []*process.PSW // процессы в состоянии «Готов»
}

// Add добавляет процесс в конец очереди готовности.
// Вызывается регулятором при переводе процесса в состояние «Готов».
func (p *Planner) Add(psw *process.PSW) {
    p.ReadyQueue = append(p.ReadyQueue, psw)
}

// Remove исключает процесс из очереди готовности.
// Вызывается регулятором при изменении состояния процесса.
func (p *Planner) Remove(psw *process.PSW) {
    for i, r := range p.ReadyQueue {
        if r == psw {
            p.ReadyQueue = append(p.ReadyQueue[:i], p.ReadyQueue[i+1:]...)
            return
        }
    }
}

// SelectNext выбирает процесс с наивысшим DynamicPriority, не изменяя
// ни очередь, ни состояния процессов.
func (p *Planner) SelectNext() *process.PSW {
    if len(p.ReadyQueue) == 0 {
        return nil
    }

    // Формула выбора: процесс с максимальным DynamicPriority;
    // при равенстве приоритетов выбирается стоящий раньше в очереди.
    best := 0
    for i, psw := range p.ReadyQueue {
        if psw.DynamicPriority > p.ReadyQueue[best].DynamicPriority {
            best = i
        }
    }
    return p.ReadyQueue[best]
}

// UpdatePriorities пересчитывает приоритеты процессов в очереди готовности.
// Вызывается каждый такт (шаг Б5 алгоритма такта).

```



```
//
// Вариант 11: прирост DynamicPriority тем больше, чем больше размер задания
// (PSW.Task.Size) – приоритет больших заданий увеличивается сильнее.
func (p *Planner) UpdatePriorities() {
    for _, psw := range p.ReadyQueue {
        // Прирост за такт ожидания пропорционален размеру задания.
        psw.DynamicPriority += psw.Task.Size
    }
}
```

Файл `internal/interrupt/interrupt.go`:

```
// Package interrupt моделирует сигналы прерывания и их обработку.
//
// Сигнал формируется источником (процессором ввода-вывода) в момент
// окончания операции и помещается в очередь сигналов (Post). Обработку
// сигналов выполняет супервизор прерываний: ядро модели извлекает сигналы
// (ExtractAll) и через регуляторщик изменяет состояния процессов.
package interrupt

import "os_model/internal/process"

// Type – тип (источник) прерывания.
type Type int

const (
    IOComplete Type = iota // окончание операции ввода-вывода
    Timer                  // прерывание от таймера (в варианте 11 не используется)
)

// Signal – смоделированный сигнал прерывания.
type Signal struct {
    Type    Type    // тип прерывания
    Time    int        // такт модели, в котором возникло прерывание
    Process *process.PSW // процесс, которого касается прерывание
    SourceID int        // номер источника (процессора ввода-вывода)
}

// logLimit – сколько последних сигналов хранится для индикации.
const logLimit = 10

// Supervisor – супервизор прерываний: принимает сигналы от источников,
// накапливает их для обработки и ведёт журнал последних прерываний.
type Supervisor struct {
    Pending []Signal // сигналы, ожидающие обработки
    Log     []Signal // журнал последних сигналов (для индикации)
    Handled int     // всего обработано сигналов
}

// Post – источник публикует сигнал прерывания.
func (s *Supervisor) Post(signal Signal) {
    s.Pending = append(s.Pending, signal)
    s.Log = append(s.Log, signal)
    if len(s.Log) > logLimit {
        s.Log = s.Log[len(s.Log)-logLimit:]
    }
}

// ExtractAll извлекает все накопившиеся сигналы для обработки.
func (s *Supervisor) ExtractAll() []Signal {
    pending := s.Pending
    s.Pending = nil
    s.Handled += len(pending)
}
```

```

    return pending
}

```

Файл `internal/io/processors.go`:

```

// Package io моделирует подсистему ввода-вывода — процессоры ВВ.
//
// Количество процессоров ввода-вывода достаточно для работы без очередей
// к ним, поэтому каждая операция выполняется на отдельном (условном)
// процессоре ВВ. По окончании операции процессор ВВ публикует сигнал
// прерывания; состояния процессов изменяет только регулятор.
package io

import (
    "os_model/internal/interrupt"
    "os_model/internal/process"
    "os_model/internal/regulator"
)

// Operation — операция ввода-вывода, выполняемая на процессоре ВВ.
type Operation struct {
    ProcessorID int // номер процессора ВВ
    Process     *process.PSW // обслуживаемый процесс
    TicksLeft   int // осталось тактов выполнения
    TotalTicks  int // полная длительность операции

    initialized bool // такт инициализации пройден
    started     bool // процесс переведён в состояние «Блокирован»
}

// Processors обслуживает процессоры ввода-вывода.
type Processors struct {
    Operations []*Operation // выполняемые операции (занятые процессоры ВВ)

    // Regulator — регулятор: единственный, кто изменяет состояния процессов.
    Regulator *regulator.Regulator

    // Interrupts — супервизор прерываний: приёмник сигналов от процессоров ВВ.
    Interrupts *interrupt.Supervisor

    nextProcessor int // счётчик номеров процессоров ВВ
}

// InitIO — подпрограмма инициализации операции ввода-вывода: выделяет
// процессор ВВ, сохраняет слово состояния процесса (через регулятора)
// и переводит его в состояние «Инициализация IO».
func (m *Processors) InitIO(p *process.PSW, duration int) {
    m.nextProcessor++

    // Смена состояния — только через регулятора.
    m.Regulator.Process(p, regulator.EventIORequest)

    m.Operations = append(m.Operations, &Operation{
        ProcessorID: m.nextProcessor,
        Process:     p,
        TicksLeft:   duration,
        TotalTicks:  duration,
    })
}

// TickIO — подпрограмма работы процессоров ввода-вывода: выполняет один такт

```

```

// каждой операции, уменьшает счётчики тактов и по окончании операции
// публикует сигнал прерывания. Состояния процессов не изменяются.
func (m *Processors) TickIO(now int) {
    busy := m.Operations[:0]
    for _, op := range m.Operations {
        // Такт, в котором операция инициализирована, целиком уходит
        // на инициализацию: процессор ВВ принимает операцию.
        if !op.initialized {
            op.initialized = true
            busy = append(busy, op)
            continue
        }

        // Первый такт обслуживания: «Инициализация IO» → «Блокирован».
        if !op.started {
            op.started = true
            m.Regulator.Process(op.Process, regulator.EventIOStart)
        }

        op.TicksLeft--
        if op.TicksLeft > 0 {
            busy = append(busy, op)
            continue
        }

        // Счётчик тактов равен нулю — процессор ВВ публикует прерывание.
        m.Interrupts.Post(interrupt.Signal{
            Type:    interrupt.IOComplete,
            Time:    now,
            Process:  op.Process,
            SourceID: op.ProcessorID,
        })
    }
    m.Operations = busy
}

```

Файл `internal/os/system.go`:

```

package os

import (
    "math/rand/v2"
    "time"

    "os_model/internal/cpu"
    "os_model/internal/interrupt"
    "os_model/internal/io"
    "os_model/internal/memory"
    "os_model/internal/process"
    "os_model/internal/regulator"
)

// Параметры модели по умолчанию (если соответствующие поля Config не заполнены).
const (
    DefaultMemorySize      = 8192 // общий объём моделируемой памяти
    DefaultTickDurationMs  = 1    // начальная длительность такта, мс
    DefaultIOMaxDuration   = 10   // максимальная длительность ввода-вывода, тактов
)

// Параметры генерации заданий (значения по умолчанию на этапе компиляции;
// на этапе 8 будут читаться из файла параметров).
const (
    TaskSizeMin = 100 // размер задания, единиц памяти

```

```

TaskSizeMax = 500

TaskCommandsMin = 10 // число команд задания
TaskCommandsMax = 20

TaskIOPercentMin = 0 // процент команд ввода-вывода
TaskIOPercentMax = 50

TaskBasePriorityMin = 1 // базовый приоритет
TaskBasePriorityMax = 10
)

// MaxProcesses – число записей в таблице слов состояний процессов.
const MaxProcesses = 16

// Границы скорости моделирования: от 0,1 до 1000 тактов в секунду.
const (
    minSpeed      = 0.1
    maxSpeed      = 1000.0
    speedStepPercent = 10
)

// Directive – управляющая директива оператора.
type Directive int

const (
    DirectivePause      Directive = iota // приостановить моделирование
    DirectiveResume      // продолжить моделирование
    DirectiveSpeedUp      // увеличить скорость
    DirectiveSpeedDown    // уменьшить скорость
    DirectiveQuit         // завершить моделирование
)

// System – ядро модели: связывает все компоненты и содержит главный цикл.
type System struct {
    Memory      *memory.Allocator // супервизор памяти
    Regulator   *regulator.Regulator // регулировщик (состояния процессов,
    планировщик)
    Interrupts  *interrupt.Supervisor // супервизор прерываний
    CPU         *cpu.CPU // центральный процессор
    IO          *io.Processors // подсистема ввода-вывода (процессоры ВВ)

    Config Config // параметры конфигурации

    Time int // глобальное время модели в тактах

    // Управление работой модели.
    Directives chan Directive // директивы оператора
    Speed      float64 // текущая скорость модели, тактов в секунду
    Paused     bool // моделирование приостановлено
    Quit       bool // получена директива завершения

    // OnTick вызывается каждый такт для индикации состояния модели
    // (например, для отправки снимка в интерфейс). Может быть nil.
    OnTick func(s *System)

    nextTaskID int // счётчик идентификаторов заданий
}

// Init инициализирует структуры модели по значениям полей Config
// до запуска главного цикла.
func (sys *System) Init() {
    // Значения по умолчанию для незаполненной (или невалидной) конфигурации.

```

```

if sys.Config.MemorySize <= 0 {
    sys.Config.MemorySize = DefaultMemorySize
}
if sys.Config.TickDurationMs <= 0 {
    sys.Config.TickDurationMs = DefaultTickDurationMs
}
if sys.Config.IOMaxDuration <= 0 {
    sys.Config.IOMaxDuration = DefaultIOMaxDuration
}

// Создание подсистем и их связывание.
sys.Memory = &memory.Allocator{TotalSize: sys.Config.MemorySize}
sys.Regulator = &regulator.Regulator{
    Planner:      &regulator.Planner{},
    Memory:       sys.Memory,
    MaxProcesses: MaxProcesses,
}
sys.Interrupts = &interrupt.Supervisor{}
sys.IO = &io.Processors{
    Regulator: sys.Regulator,
    Interrupts: sys.Interrupts,
}
sys.CPU = &cpu.CPU{
    IO:          sys.IO,
    Processes:   sys.Regulator,
    IOMaxDuration: sys.Config.IOMaxDuration,
}

if sys.Directives == nil {
    sys.Directives = make(chan Directive, MaxProcesses)
}

sys.Time = 0
sys.nextTaskID = 0
sys.Paused = false
sys.Quit = false
sys.setSpeed(1000.0 / float64(sys.Config.TickDurationMs))
}

// GenerateAndLoadTask – системный процесс загрузки задания: генерирует
// параметры нового задания и пытается загрузить его, если есть ресурсы
// (свободная запись в таблице слов состояний процессов и достаточная
// память). При успехе регулировщик создаёт процесс в состоянии «Готов».
// Возвращает true, если задание загружено.
func (sys *System) GenerateAndLoadTask() bool {
    // Проверка места в таблице слов состояний процессов.
    if !sys.Regulator.FreeSlot() {
        return false
    }

    // Генерируем параметры задания и проверяем наличие доступной памяти
    // (новое задание генерируется только при наличии свободной памяти).
    task := sys.generateTask()
    if !sys.Memory.CheckAvailable(task.Size) {
        return false
    }

    // Присваиваем идентификатор и выделяем память для задания.
    sys.nextTaskID++
    task.ID = sys.nextTaskID
    if err := sys.Memory.Allocate(task); err != nil {
        return false
    }

    // Порождаем процесс; состояние «Готов» устанавливает регулировщик.
    psw := &process.PSW{
        ID:          task.ID,

```

```

        Task:          &task,
        PC:            0,
        DynamicPriority: task.BasePriority,
    }

    return sys.Regulator.Process(psw, regulator.EventLoad)
}

// HandleDirectives обрабатывает директивы оператора:
// пауза, изменение скорости моделирования, завершение работы.
func (sys *System) HandleDirectives() {
    for {
        select {
        case d := <-sys.Directives:
            sys.applyDirective(d)
        default:
            return
        }
    }
}

// RunLoop – главный цикл модели. В каждом такте выполняется подпрограмма
// DoOneTick (алгоритм одного такта моделирования), затем обрабатываются
// директивы оператора и обновляется индикация.
func (sys *System) RunLoop() {
    // Цикл загрузки заданий для начала моделирования: загружается столько
    // заданий, сколько помещается в память и в таблицу процессов.
    for sys.GenerateAndLoadTask() {

        for !sys.Quit {
            if !sys.Paused {
                sys.DoOneTick()
            } else {
                // На паузе изменения (директивы, скорость) тоже нужно показывать.
                sys.notify()
            }

            sys.HandleDirectives()

            if sys.Quit {
                sys.notify() // завершающий снимок состояния
                break
            }

            sys.sleep()
        }
    }

    // DoOneTick – выполнение одного такта моделирования в соответствии
    // с алгоритмом одного такта модели ОС:
    //
    // А – инициализация цикла: выбрать процесс, сделать его активным
    // и восстановить слово состояния процесса;
    // Б1 – моделирование ЦПр: выполнить очередную команду активного процесса;
    // Б2 – моделирование процессоров ВВ: один такт операций ввода-вывода
    // и обработка прерываний от процессоров ВВ;
    // Б3 – моделирование прерываний по времени (только для алгоритма RR;
    // вариант 11 использует относительные приоритеты без вытеснения,
    // поэтому шаг отсутствует);
    // Б4 – проверка достигнутого состояния системы и вызов планировщика;
    // Б5 – обновление баз данных модели ОС;
    // Б6 – отображение изменений в состоянии системы.
    func (sys *System) DoOneTick() {
        sys.Time++
    }
}

```

```

// А. Инициализация цикла.
if sys.CPU.State == cpu.StateIdle {
    sys.dispatchProcess()
}

// Б1. Моделирование центрального процессора.
sys.CPU.Tick()

// Б2. Моделирование процессоров ввода-вывода и обработка прерываний.
sys.IO.TickIO(sys.Time)
sys.HandleInterrupts()

// Б3. Прерывания по времени не моделируются (см. комментарий выше).

// Б4. Проверка достигнутого состояния системы.
sys.CheckSystemState()

// Б5. Обновление баз данных модели ОС.
sys.Regulator.Planner.UpdatePriorities()

// Б6. Отображение изменений в состоянии системы.
sys.notify()
}

// dispatchProcess – выбор планировщиком следующего процесса и передача его
// на выполнение: регулятор переводит процесс в состояние «Активен»,
// процессор восстанавливает слово состояния процесса.
func (sys *System) dispatchProcess() bool {
    p := sys.Regulator.Planner.SelectNext()
    if p == nil {
        return false
    }

    if !sys.Regulator.Process(p, regulator.EventActivate) {
        return false
    }

    sys.CPU.ContextSwitch(p)
    return true
}

// CheckSystemState – проверка достигнутого состояния системы (шаг Б4).
//
// Для всех процессоров: если резидентный процесс перестал быть активным
// (заблокирован или завершён), процессор освобождается; если процессор
// находится в состоянии «Ожидание», а список готовности не пуст, вызывается
// планировщик для выбора следующего процесса. Если готовых процессов нет,
// процессор переходит в состояние «Ожидание», а ОС порождает системный
// процесс загрузки нового задания, если есть ресурсы.
func (sys *System) CheckSystemState() {
    // Резидентный процесс мог перестать быть активным: его состояние
    // изменил регулятор по событию ВВ или завершения.
    if p := sys.CPU.ActiveProcess; p != nil && p.State != process.StateActive {
        sys.CPU.Release()
    }

    if sys.CPU.State != cpu.StateIdle {
        return
    }

    // Список готовности не пуст – планировщик выбирает следующий процесс.
    if len(sys.Regulator.Planner.ReadyQueue) > 0 {
        sys.dispatchProcess()
        return
    }
}

```

```

    }

    // Готовых процессов нет: ЦПр «Ожидает», ОС может загрузить одно новое
    // задание (место в таблице PSW и достаточная память проверяются
    // в GenerateAndLoadTask).
    if sys.GenerateAndLoadTask() {
        sys.dispatchProcess()
    }
}

// HandleInterrupts – супервизор прерываний: обрабатывает накопленные сигналы.
// Обработка сводится к вызову регуляровщика, который изменяет состояния
// процессов (прерывание от процессора ВВ переводит процесс в «Готов»).
func (sys *System) HandleInterrupts() {
    for _, signal := range sys.Interrupts.ExtractAll() {
        switch signal.Type {
        case interrupt.IOComplete:
            sys.Regulator.Process(signal.Process, regulator.EventIOComplete)
        }
    }
}

// generateTask генерирует параметры нового задания в заданных диапазонах
// (без идентификатора: идентификатор присваивается загруженному заданию).
func (sys *System) generateTask() process.Task {
    return process.Task{
        Size:      randomInRange(TaskSizeMin, TaskSizeMax),
        TotalCommands: randomInRange(TaskCommandsMin, TaskCommandsMax),
        IOPercent:  randomInRange(TaskIOPercentMin, TaskIOPercentMax),
        BasePriority: randomInRange(TaskBasePriorityMin, TaskBasePriorityMax),
    }
}

// applyDirective выполняет одну директиву оператора.
func (sys *System) applyDirective(d Directive) {
    switch d {
    case DirectivePause:
        sys.Paused = true
    case DirectiveResume:
        sys.Paused = false
    case DirectiveSpeedUp:
        sys.changeSpeed(speedStepPercent)
    case DirectiveSpeedDown:
        sys.changeSpeed(-speedStepPercent)
    case DirectiveQuit:
        sys.Quit = true
    }
}

// changeSpeed изменяет скорость модели на заданное число процентов.
func (sys *System) changeSpeed(percent int) {
    sys.setSpeed(sys.Speed * float64(100+percent) / 100)
}

// setSpeed устанавливает скорость модели (тактов в секунду)
// в допустимых пределах.
func (sys *System) setSpeed(speed float64) {
    if speed < minSpeed {
        speed = minSpeed
    }
    if speed > maxSpeed {
        speed = maxSpeed
    }
    sys.Speed = speed
}

// notify вызывает обработчик индикации, если он установлен.

```



```

func (sys *System) notify() {
    if sys.OnTick != nil {
        sys.OnTick(sys)
    }
}

// sleep выполняет программную задержку в соответствии со скоростью модели.
func (sys *System) sleep() {
    if sys.Speed <= 0 {
        return
    }
    time.Sleep(time.Duration(float64(time.Second) / sys.Speed))
}

// randomInRange возвращает случайное число из диапазона [min, max].
func randomInRange(min, max int) int {
    if max <= min {
        return min
    }
    return min + rand.IntN(max-min+1)
}

```

Файл `internal/os/snapshot.go`:

```

package os

import (
    "os_model/internal/interrupt"
    "os_model/internal/io"
    "os_model/internal/process"
)

// ProcessInfo – данные одного процесса для индикации.
type ProcessInfo struct {
    ID          int `json:"id"`
    Size        int `json:"size"`
    PC          int `json:"pc"`
    TotalCommands int `json:"totalCommands"`
    IOPercent   int `json:"ioPercent"`
    State       int `json:"state"`
    DynamicPriority int `json:"dynamicPriority"`
    BasePriority int `json:"basePriority"`
}

// OperationInfo – данные одной операции ввода-вывода (занятого процессора ВВ).
type OperationInfo struct {
    ProcessorID int `json:"processorId"` // номер процессора ВВ
    ProcessID   int `json:"processId"`   // обслуживаемый процесс
    State       int `json:"state"`       // «Инициализация IO» или «Блокирован»
    TicksLeft   int `json:"ticksLeft"`   // осталось тактов
    TotalTicks  int `json:"totalTicks"`  // полная длительность операции
}

// InterruptInfo – данные сигнала прерывания для индикации.
type InterruptInfo struct {
    Type      int `json:"type"` // тип прерывания
    Time      int `json:"time"` // такт модели
    ProcessID int `json:"processId"` // процесс, которого касается прерывание
    SourceID  int `json:"sourceId"` // номер источника (процессора ВВ)
}

// TaskParams – диапазоны генерации заданий (входные параметры модели).

```

```

type TaskParams struct {
    SizeMin      int `json:"sizeMin"`
    SizeMax      int `json:"sizeMax"`
    CommandsMin  int `json:"commandsMin"`
    CommandsMax  int `json:"commandsMax"`
    IOPercentMin int `json:"ioPercentMin"`
    IOPercentMax int `json:"ioPercentMax"`
    PriorityMin   int `json:"priorityMin"`
    PriorityMax   int `json:"priorityMax"`
}

// Snapshot – снимок состояния модели для индикации в интерфейсе.
type Snapshot struct {
    Time      int    `json:"time"`
    Speed     float64 `json:"speed"`
    Paused    bool    `json:"paused"`
    Quit      bool    `json:"quit"`
    Generated int    `json:"generated"`
    Completed int    `json:"completed"`

    MemoryTotal int `json:"memoryTotal"`
    MemoryUsed  int `json:"memoryUsed"`

    CPUState      int `json:"cpuState"` // 0 – ожидание, 1 – работа
    ActiveID      int `json:"activeId"`  // 0 – нет активного процесса
    CommandType   int `json:"commandType"` // 0 – вычислительная, 1 – IO, 2 –
завершение
    CommandCode   int `json:"commandCode"` // код вычислительной операции
    CommandAddr1  int `json:"commandAddr1"` // адрес первого операнда
    CommandAddr2  int `json:"commandAddr2"` // адрес второго операнда
    CommandDuration int `json:"commandDuration"` // длительность операции IO,
тактов

    // Прерывания: счётчик обработанных сигналов и журнал последних.
    InterruptsHandled int `json:"interruptsHandled"`
    InterruptLog      []InterruptInfo `json:"interruptLog"`

    MaxProcesses int `json:"maxProcesses"`

    Ready      []ProcessInfo `json:"ready"` // очередь готовности
    Operations []OperationInfo `json:"operations"` // занятые процессоры BB
    Processes  []ProcessInfo `json:"processes"` // таблица PSW

    TaskParams TaskParams `json:"taskParams"`
}

// Snapshot формирует снимок текущего состояния модели.
func (sys *System) Snapshot() Snapshot {
    snap := Snapshot{
        Time:      sys.Time,
        Speed:     sys.Speed,
        Paused:    sys.Paused,
        Quit:      sys.Quit,
        Generated: sys.nextTaskID,
        Completed: sys.Regulator.Completed,
        MemoryTotal: sys.Memory.TotalSize,
        MemoryUsed:  sys.Memory.UsedSize,
        CPUState:    int(sys.CPU.State),
        InterruptsHandled: sys.Interrupts.Handled,
        MaxProcesses: MaxProcesses,
        TaskParams: TaskParams{
            SizeMin:      TaskSizeMin,
            SizeMax:     TaskSizeMax,
            CommandsMin: TaskCommandsMin,
            CommandsMax: TaskCommandsMax,
            IOPercentMin: TaskIOPercentMin,

```

```

        IOPercentMax: TaskIOPercentMax,
        PriorityMin: TaskBasePriorityMin,
        PriorityMax: TaskBasePriorityMax,
    },
}

// Команда актуальна только при активном процессе.
if p := sys.CPU.ActiveProcess; p != nil {
    snap.ActiveID = p.ID
    snap.CommandType = int(sys.CPU.CurrentCommand.Type)
    snap.CommandCode = int(sys.CPU.CurrentCommand.Code)
    snap.CommandAddr1 = sys.CPU.CurrentCommand.Address1
    snap.CommandAddr2 = sys.CPU.CurrentCommand.Address2
    snap.CommandDuration = sys.CPU.CurrentCommand.Duration
}

snap.Ready = processInfos(sys.Regulator.Planner.ReadyQueue)
snap.Operations = operationInfos(sys.IO.Operations)
snap.Processes = processInfos(sys.Regulator.Table)
snap.InterruptLog = interruptInfos(sys.Interrupts.Log)
return snap
}

// processInfos преобразует список PSW в данные для индикации.
func processInfos(psws []*process.PSW) []ProcessInfo {
    infos := make([]ProcessInfo, 0, len(psws))
    for _, p := range psws {
        infos = append(infos, ProcessInfo{
            ID:          p.ID,
            Size:        p.Task.Size,
            PC:          p.PC,
            TotalCommands: p.Task.TotalCommands,
            IOPercent:    p.Task.IOPercent,
            State:        int(p.State),
            DynamicPriority: p.DynamicPriority,
            BasePriority:  p.Task.BasePriority,
        })
    }
    return infos
}

// operationInfos преобразует операции ввода-вывода в данные для индикации.
func operationInfos(operations []*io.Operation) []OperationInfo {
    infos := make([]OperationInfo, 0, len(operations))
    for _, op := range operations {
        infos = append(infos, OperationInfo{
            ProcessorID: op.ProcessorID,
            ProcessID:  op.Process.ID,
            State:       int(op.Process.State),
            TicksLeft:   op.TicksLeft,
            TotalTicks: op.TotalTicks,
        })
    }
    return infos
}

// interruptInfos преобразует журнал прерываний в данные для индикации.
func interruptInfos(signals []interrupt.Signal) []InterruptInfo {
    infos := make([]InterruptInfo, 0, len(signals))
    for _, s := range signals {
        info := InterruptInfo{
            Type:    int(s.Type),
            Time:    s.Time,
            SourceID: s.SourceID,
        }
        if s.Process != nil {
            info.ProcessID = s.Process.ID
        }
    }
}

```

```
        infos = append(infos, info)
    }
    return infos
}
```

Результат

В соответствии с индивидуальным заданием (вариант 11 — относительные и динамические приоритеты) получены следующие результаты.

Формула выбора следующего процесса для обслуживания на ЦПр — процесс с наибольшим динамическим приоритетом:

$$P = \operatorname{argmax}(\operatorname{DynamicPriority}),$$

выбор выполняется только при освобождении ЦПр (относительные приоритеты — активный процесс не вытесняется). Динамический приоритет процесса, находящегося в очереди готовности, увеличивается каждый такт:

$$\operatorname{DynamicPriority} = \operatorname{DynamicPriority} + \operatorname{Size},$$

где Size — размер задания в единицах памяти: чем больше задание, тем сильнее растёт его приоритет. Индикация параметров алгоритма планировщика приведена в интерфейсе модели: в очереди готовности отображаются текущие динамические приоритеты процессов с полосами относительного уровня, а формулы вынесены в подвал окна.

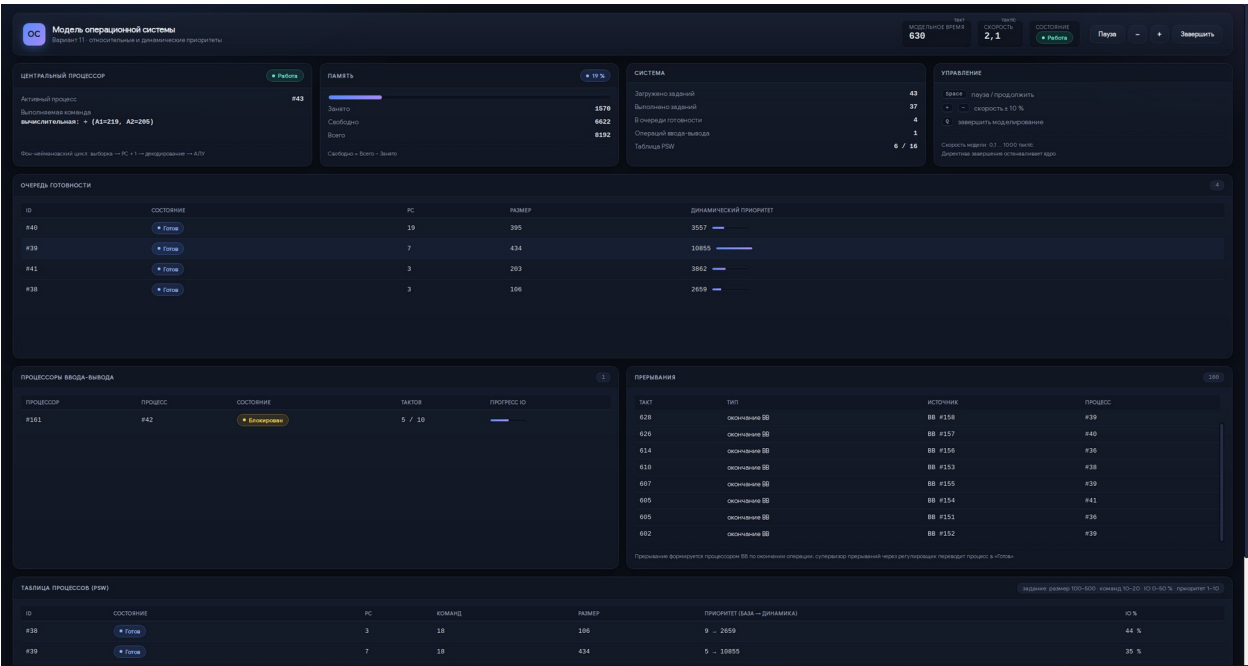


Рисунок 5 — Общий вид модели ОС

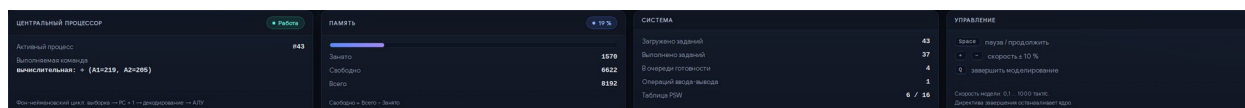


Рисунок 6 — Состояние ЦПр, памяти и системы (загружено/выполнено заданий, операции ВВ)

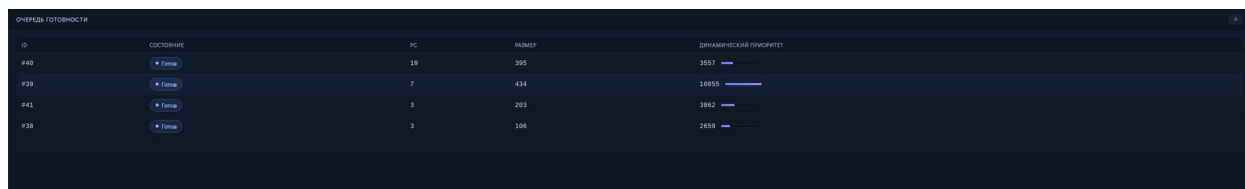


Рисунок 7 — Очередь готовности: динамические приоритеты процессов (вариант 11)

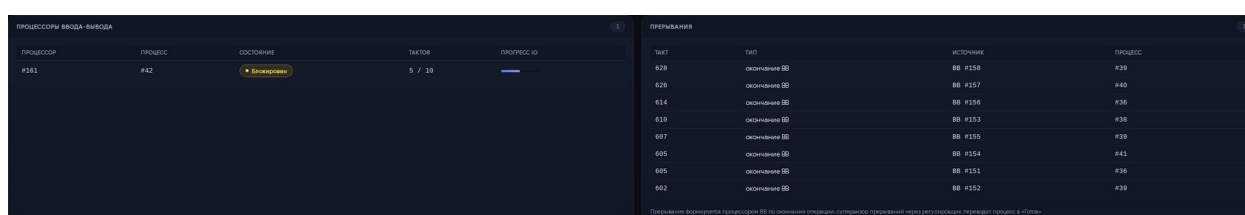


Рисунок 8 — Процессоры ввода-вывода и журнал сигналов прерываний

На рисунке 7 видно, что процессы в очереди готовности имеют разные динамические приоритеты: чем дольше процесс ожидает и чем больше его размер, тем выше его приоритет (например, процесс #39 с размером 434 имеет приоритет 10855, а процесс #38 с размером 106 — только 2659). На рисунке 8 показан занятый процессор ВВ, обслуживающий операцию процесса #42 (состояние «Блокирован», осталось 5 тактов из 10), и журнал прерываний: по окончании каждой операции процессор ВВ публикует сигнал, супервизор прерываний через регулировщик возвращает процесс в состояние «Готов».

Вывод

В ходе данной работы были реализованы метод обработки прерываний и регулировщик применительно к ПО «Модель ОС». Состояния процессов изменяет только регулировщик — подпрограмма Process; центральный процессор, подсистема ввода-вывода и планировщик лишь сообщают о событиях. Реализованы подпрограмма работы процессоров ввода-вывода,

моделирование сигнала прерывания по окончании операции ВВ, проверка состояния системы, а также обновлены подпрограмма выполнения одного такта моделирования и подпрограмма индикации. Получены формулы выбора следующего процесса для обслуживания на ЦПр и индикация параметров индивидуального алгоритма планировщика (вариант 11 — относительные и динамические приоритеты). Получены практические навыки построения регуляровщика и обработки прерываний в модели операционной системы.